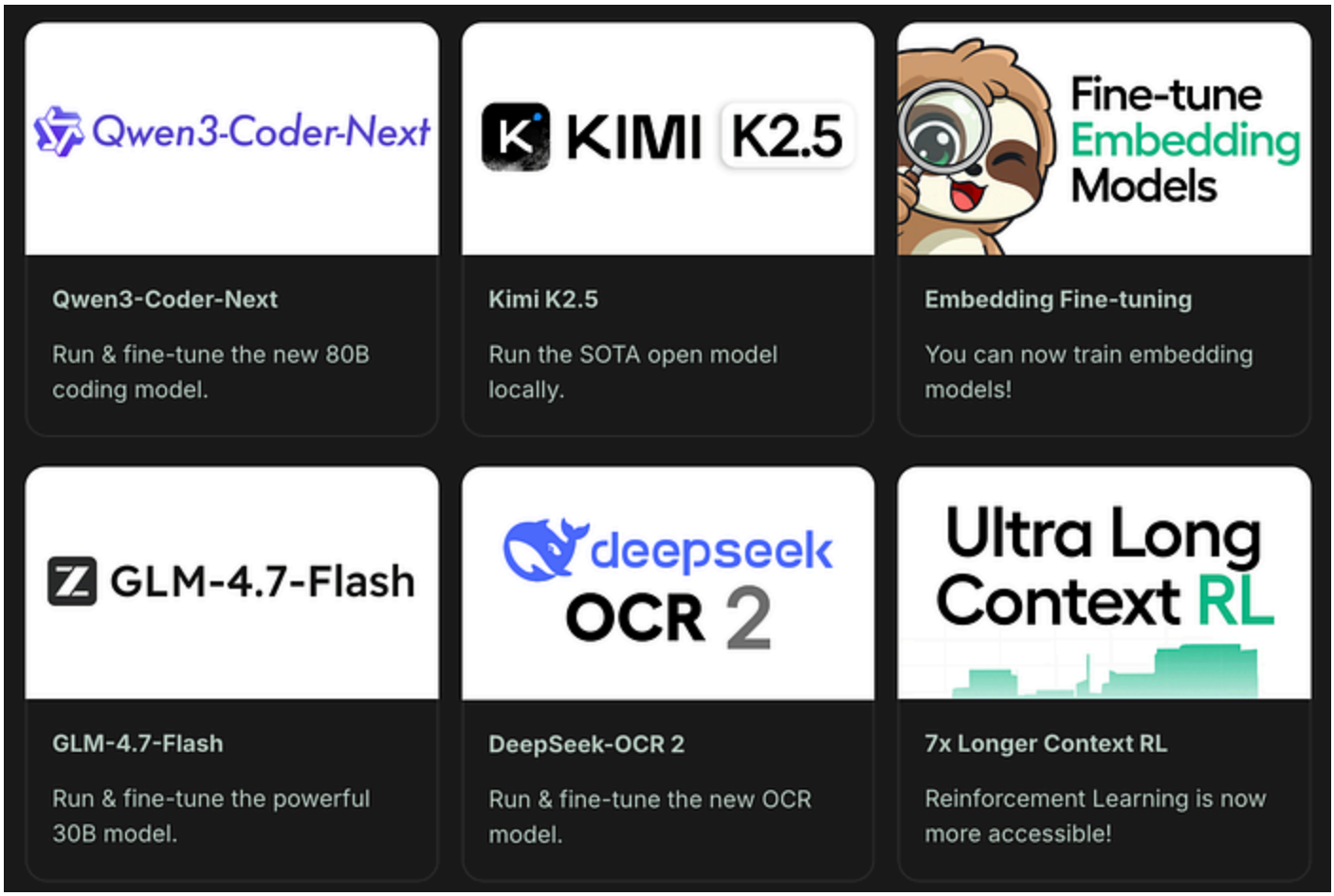
Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



Developer's Deep Dive Into Fine-Tuning Modern LLMs

How well fine-tuning performs still depends on three factors: model size, hardware capability, and the framework you choose.

Agent Native Follow
~19 min read · February 8, 2026 (Updated: February 8, 2026) · Free: No

Let's talk numbers.

Most modern open-source models in the 7–8B parameter range (like LLaMA 3, Mistral, or Qwen3) can now be fine-tuned on GPUs with **12–24 GB VRAM** using **4-bit quantized adapters (QLoRA)**.

An 8B model typically needs just **5–6 GB of memory,** making it feasible on a **RTX 3060** or newer iterations.

For memory efficiency, dynamic quantization narrowed the accuracy gap such that using 4-bit has minimal downside.

QLoRA vs LoRA has negligible loss now, meaning you can confidently use 4-bit to save memory. You should train in the precision you plan to serve in (so if you want a 4-bit runtime model, train in 4-bit).

On speed, with optimizations like FlashAttention, LoRA, and sequence packing, you can fine-tune 8B model on 100k samples in few hours on an A100 (40GB), whereas the same on a consumer RTX 4090 (~24GB, slightly less memory and compute) might take longer (~10–12 hours) with these optimizations, and ~20+ hours on a 3090. Without the optimizations that these frameworks provide, it can take 2–3x longer.

If you have access to multiple-GPUs (e.g. two 3090s), you can split the load almost linearly. Fine-tuning a 13B model might go nearly 2x faster on 2 GPUs versus 1.

If you have Apple Silicon, to my knowledge, **7B and 13B fine-tuning is technically possible**, but training is **5–10x slower** due to the lack of Metal-optimized low-level kernels. If you're on a MacBook with an M1 or higher, stick to 3B–7B models and limit to a few epochs, or offload training to a cloud GPU (like RunPod or Lambda). That said, I'm not an expert there, so I'd love to hear about your Mac/M-series experiences in the comments.

On the integrations side, all frameworks allow saving models in Hugging Face format, so you can easily push to the Hub or load with `AutoModel`.

In this guide, I will walk you through four leading solutions,

- **Unsloth:** low-VRAM, 2x faster training, beginner-friendly notebooks
- **LLaMA-Factory:** full-stack CLI + Web UI, 100+ models, multi-modal
- **DeepSpeed:** battle-tested scaling for large models and multi-GPU
- **Axolotl:** YAML-first, reproducible, research-grade tuning workflows

We'll explore features and practical examples (code, CLI, YAML) of each toolkit.

Overview of Fine-Tuning Frameworks

To set the stage, here's a high-level comparison of the four frameworks:

Framework	Approach & Ease of Use	Hardware Needs	Fine-Tuning Methods	Notable Features
Unsloth	Python library (notebook-centric); no-code Colab notebooks available for quick start. Simple API for local use via pip install.	Minimal GPU memory – e.g. fine-tune a 4B model on 3 GB VRAM; consumer GPUs supported (NVIDIA/Windows, Linux).	Full FT, LoRA/QLoRA, RLHF (GRPO), embedding & TTS fine-tunes.	Performance optimized: ~2× faster training with 70%+ less VRAM usage (4-bit quantization); dynamic 4-bit quant (recovering QLoRA accuracy); 4× context length fine-tuning; multimodal support (vision, speech).
LLaMA-Factory	Comprehensive zero-code toolkit with both CLI and a Web UI ("LLaMA Board") for fine-tuning. Easy for beginners via GUI; advanced users can edit YAML configs.	Moderate GPU – e.g. 7B–13B models on a single high-VRAM GPU (≈16–24 GB) using 4-bit LoRA. Multi-GPU and even AMD/Ascend support (ROCm, NPUs) provided.	Full FT, Freeze tuning, LoRA/QLoRA (2–8 bit adapters), Reward tuning (PPO/DPO), multi-modal SFT (supports image/text models).	100+ models supported out-of-the-box (LLaMA 3/4, Mistral, Qwen3, Gemma, etc.); Day-0 updates for new models; advanced research optimizations (LongLoRA, LoRA+, MuZero optimizers); rich monitoring (LlamaBoard UI, TensorBoard, W&B); integration with vLLM for fast inference.
DeepSpeed	Low-level library by Microsoft – integrates with PyTorch/Hugging Face Trainer rather than a standalone UI. Configured via JSON or accelerate YAML. Usability requires some engineering, but Hugging Face Accelerate provides a simple interface for it.	Scales from one GPU to multi-node clusters. Memory optimization (ZeRO) enables training models larger than a single GPU's RAM by sharding states across GPUs. On one GPU, can offload to CPU/NVMe (trading speed for capacity).	Full FT (with mixed precision FP16/BF16/FP8), supports LoRA/QLoRA via PEFT integration (not built-in but used alongside). Suited for distributed and large-scale fine-tuning.	ZeRO optimizations (stages 1–3) for memory efficiency; 15× speedups reported for RLHF pipelines vs. baseline; supports FP8 training on new GPUs (H100/Mi300); multi-node and multi-GPU training with ease (via Accelerate launcher). Widely used for training 70B+ models with quantization.
Axolotl	Config-driven open-source toolkit – define training runs in a single YAML file, then run via CLI. No built-in GUI (uses code or CLI). Designed to be beginner-friendly yet highly customizable.	Flexible: single-GPU (consumer cards) or multi-GPU. Requires GPU for reasonable speed (M-series Mac support is partial; CPU/MPS runs are possible but limited). Supports NVIDIA (CUDA) fully; also AMD ROCm and multi-node (via Ray/torchrun).	Full FT, LoRA/QLoRA (with memory optimizations), GPTQ (quantized training), QAT (quantization-aware FT), RLHF (GRPO/ORPO/PPO), Reward modeling, even text-to-image diffusion fine-tunes.	Multiple model families (LLaMA, Mistral, Qwen, GPT-OSS, Gemma, etc.) in one framework; multimodal training (vision-language, audio) support; YAML pipeline covers data prep, training, eval, and inference in one config; many built-in optimizations – FlashAttention, Xformers, Sequence Packing ("multipack") to remove padding overhead, etc.; supports FP8 fine-tuning (via PyTorch torch.ao backend). Provides Docker images for easy setup.

Each tool targets a slightly different audience and use-case:

- **Unsloth** focuses on *speed and minimal VRAM*, making fine-tuning accessible on modest hardware. It streamlines typical training flows (providing Colab notebooks, simple API) and achieves big efficiency gains (e.g. packing sequences so no GPU time is wasted on padding).
- **LLaMA-Factory** is a *full-stack fine-tuning toolkit*, emphasizing ease-of-use through a polished Web UI and quick support for the latest models and research tricks. It's great for those who want a one-stop solution with little coding.
- **DeepSpeed** is a *powerful optimization library* rather than an end-to-end solution. It shines when you need to train very large models or maximize performance on multi-GPU setups, but it has a steeper learning curve (it's often used under the hood in libraries like Hugging Face Accelerate).
- **Axolotl** hits a middle ground: it's *open-source and scriptable* with simple YAML configs, ideal for developers who want fine control and the ability to run custom training routines (including cutting-edge techniques) without writing a training loop from scratch. It's a community-driven project balancing usability with flexibility.

Below, we dive into each tool in detail, with examples and recent updates.

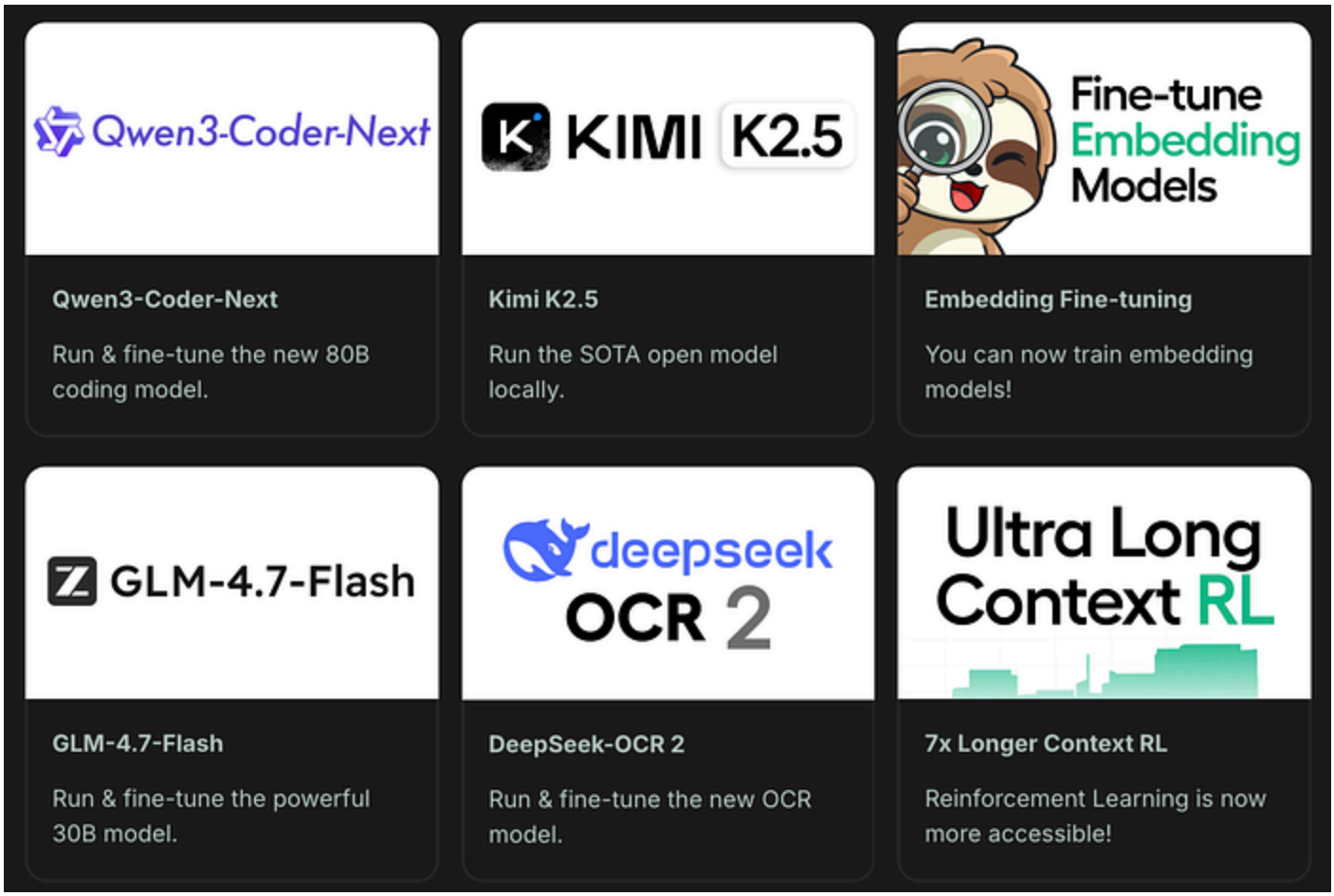
> If you're serious about building agentic products with AI, I'm writing a deep-dive eBook on **Agentic SaaS,** the emerging design patterns that are quietly powering the most innovative startups of 2026.
>
> **Grab the first chapter free:** [Agentic SaaS Patterns Winning in 2026](), and let me know what would you like to learn more about in upcoming chapters.

Unsloth: Lightweight Fine-Tuning with Low VRAM

Unsloth has quickly gained popularity for its ability to fine-tune LLMs *ultra-efficiently*.

It advertises training **"2x faster with ~70% less VRAM"** usage than standard methods, thanks to techniques like 4-bit quantization and smart data processing.

In fact, Unsloth's approach is so memory-light that one can fine-tune a 4B model (e.g. Qwen3-4B) on **just 3 GB GPU VRAM,** effectively bringing LLM fine-tuning to consumer laptops.



Unsloth is provided as a Python package (`pip install unsloth`) and a set of ready-made Jupyter notebooks.

Beginners can jump in by running Colab notebooks that guides through the whole fine-tuning process.

For instance with a single notebook you can fine-tune or do RLHF on Colab or Kaggle "for free," requiring only a few gigabytes of GPU memory.

This lowers the entry barrier significantly.

Advanced users can use Unsloth's Python API in their own scripts.

There's no dedicated GUI, but the well-documented notebooks and simple function calls make the process straightforward.

Fine-Tuning Methods

Unsloth supports standard supervised fine-tuning (SFT) as well as Reinforcement Learning fine-tuning (for RLHF-style training with reward models).

It is compatible with parameter-efficient techniques, notably **LoRA** and **QLoRA**.

Under the hood, Unsloth can apply 4-bit quantization to the base model weights (using bitsandbytes) so that you fine-tune in 4-bit mode.

This is why Unsloth can handle larger models on small GPUs.

For example, **QLoRA** reduces memory by 4x, and Unsloth further refines this with its **dynamic 4-bit quantization**.

The "dynamic" part refers to Unsloth's custom 4-bit weight quantization that preserves more accuracy than standard 4-bit methods (closing the gap between QLoRA and full 16-bit fine-tuning).

If you prefer, you can also do full 16-bit fine-tuning or 8-bit modes, Unsloth exposes flags like `load_in_4bit=True/False` or `full_finetuning=True` to toggle these modes (only one mode can be active at a time).

You are better to start with LoRA/QLoRA for most cases, since it achieves almost the same result as full fine-tuning at a fraction of the compute cost.

It also supports extended context training: with the right configuration, you can fine-tune models to **4x longer context lengths** than normal (e.g. enabling 8192-token context for LLaMA by adjusting `max_seq_length`, thanks to FlashAttention support).

Beyond text LLMs, Unsloth can fine-tune other model types: e.g. it has modules for **vision models**, text-to-speech models, embeddings, etc., making it a broadly capable training library.

Performance

A recent update introduced ***sequence packing***, which merges multiple short training samples into one sequence with proper masking, eliminating pad token overhead.

In datasets with many short prompts, this can theoreticaly give up to a **5x training speedup**.

Even in default mode without packing, the custom fused kernels and optimizations yield about **1.1–2x faster** training step times than earlier implementations.

Unsloth's **VRAM usage** can drop by **30–90%** for the same model compared to naive fine-tuning, for example, a 70B model fine-tune on a single 48 GB GPU where it would previously require multiple GPUs or not fit at all.

All this comes with "0% change in training loss curve," i.e. no accuracy degradation. These claims are also backed by the library's benchmarks and community testing.

In practice, one trade-off for using Unsloth is that it abstracts a lot for you, which is great for ease but means less manual control over training loops compared to frameworks like Axolotl.

Also, since it relies on PyTorch and NVIDIA tooling (like bitsandbytes), full support on Apple Silicon or AMD GPUs isn't its focus (running on those might fall back to slower methods or not work).

But on an NVIDIA GPU, Unsloth is one of the fastest ways to get a fine-tuned LLM.

Fine-Tuning Example with Unsloth

With Unsloth, you typically write a short Python script or use their notebook.

For instance, using Unsloth's high-level API, you might do something like:

```
from unsloth import SFTTrainer, AutoModelForCausalLM
model = AutoModelForCausalLM.from_pretrained("unsloth/llama-3.1-8b-unsloth-bnb-4bit")
```

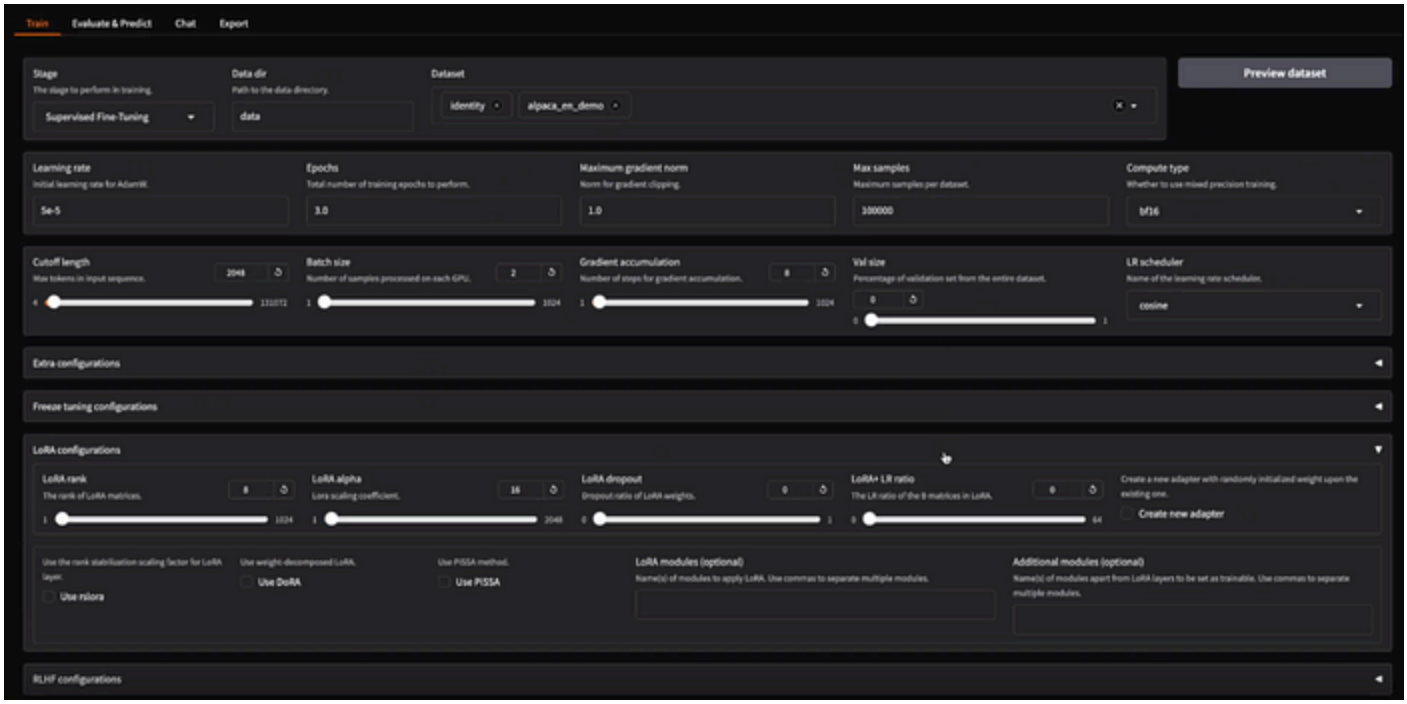
```
trainer = SFTTrainer(
    model=model,
    train_dataset=my_dataset,
    lora_rank=8,   # using LoRA rank 8
    load_in_4bit=True,
    output_dir="output-model"
)

trainer.train()
```

The key idea is that you load an Unsloth-prepared 4-bit version of a model (here LLaMA 3 8B), specify LoRA or full FT, and call `train()`.

You can [dive deeper into fine-tuning with Unsloth here](#).

LLaMA-Factory: Unified Fine-Tuning with CLI & Web UI

[LLaMA-Factory](#) is a one-stop platform for fine-tuning LLMs, combining an easy **WebUI**, a CLI tool, and a kitchen-sink of advanced features under the hood.



Its mission is to make fine-tuning *"as easy as… click Start"* while still allowing experts to leverage cutting-edge research.

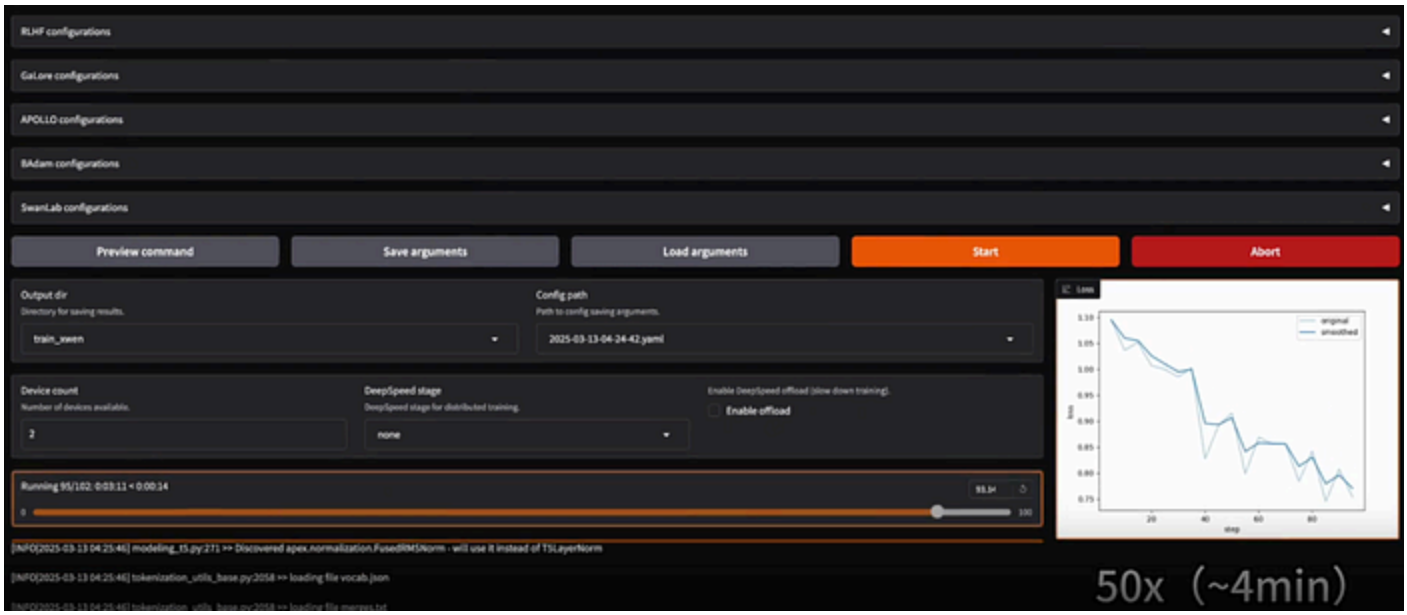
The project is notable for being very actively maintained (often adding support for new models or techniques within days, they jokingly highlight "Day-0" or "Day-1" support for models like Qwen3, Llama 4, etc.).

Usability

There are two primary ways to use LLaMA-Factory:

- **Web UI (LLaMA Board):** After installation, you launch it via `llamafactory-cli webui`, which opens a local Gradio-based web interface. This UI dramatically lowers the barrier, i.e. you can fine-tune a model without writing any code at all. Once training is done, the Evaluate/Chat tabs let you test

the model's performance right in the browser, and the Export tab lets you package the resulting model (with options to merge LoRA weights, quantize the model to GGUF/AWQ etc., and push to hub or save locally).



- **CLI & YAML:** Alternatively, everything can be done via command-line for automation. LLaMA-Factory uses YAML config files to specify training runs (similar to Axolotl). For example, to fine-tune a Qwen-3 model with LoRA, one could run:

```
llamafactory-cli train examples/train_lora/qwen3_lora_sft.yaml
```

This single command will initiate training as per the YAML config (which contains all the settings).

Likewise, there are subcommands for chatting/inference and for merging adapters.

For instance, after training, you might run `llamafactory-cli chat <config.yaml>` to launch an interactive chat with the fine-tuned model, or `llamafactory-cli export <config.yaml>` to merge LoRA weights into the base model.

The CLI is zero-code in the sense that you just prepare config files; it's suitable for those who prefer scriptable interactions or want to integrate fine-tuning into a pipeline.

Installation is a bit more involved than Unsloth or Axolotl because LLaMA-Factory has many optional components (for various hardware and features).

It supports **NVIDIA GPUs** (with CUDA), **AMD GPUs** (ROCm) and even **Huawei Ascend NPUs,** which is rare.

Depending on your hardware, you may install additional packages (e.g. for Windows, install bitsandbytes manually, for AMD, compile a custom bitsandbytes fork).

In short, it's cross-platform but NVIDIA Linux is the smoothest path.

Factory supports an **assembly line of fine-tuning approaches**:

- It covers **standard fine-tuning** (updating all model weights),

- **Freeze-tuning** (where you freeze some layers and only train certain parts), and various adapter-based tuning.
- It natively supports **LoRA** and **QLoRA**; in fact, it implements multiple quantization approaches (you can choose from 4-bit backends like `bitsandbytes` NF4, or other experimental 3-bit/2-bit methods such as AWQ or AQLM). You can do 2/3/4/6/8-bit quantized fine-tuning, which is cutting-edge, even 2-bit LoRA (though likely with some trade-offs).
- **Multi-modal:** LLaMA-Factory can fine-tune vision-language models (like LLaVA-1.5, which adds visual understanding to LLaMA) and presumably audio models as well. This means if you have an image+text dataset (for say a VQA task), the WebUI will present fields to attach image feature extraction, etc. It's quite comprehensive.
- **Reinforcement and Instruction Tuning:** It supports reward modeling and RL fine-tuning approaches such as PPO (Proximal Policy Optimization for RLHF), DPO (Direct Preference Optimization), and others like ORPO/KTO.
- **Advanced optimizers and tricks:** The toolkit integrates many research optimizations: e.g. **Apollo** and **Lion** optimizers, techniques like **LongLoRA** (for better long-sequence training), **DoRA** (perhaps a variant of LoRA), **LoftQ**, **GaLORE**, and a method called **PiSSA** (which likely relates to structured sparse adapters).
- **Monitoring and Serving:** During training, you can use built-in monitors: it supports logging to **TensorBoard**, **Weights & Biases**, MLflow, etc., as well as its native *LlamaBoard*.
- For deployment, LLaMA-Factory can even serve the model: it provides an **OpenAI-style API server** for your fine-tuned model and integrates with **vLLM** for optimized inference.
- So after fine-tuning, you could spin up a local API that serves the model (with acceleration for fast multi-threaded generation).
- **Performance:** LLaMA-Factory's efficiency is high, though perhaps a tad behind Unsloth's laser-focused optimizations. It does use **FlashAttention-2**, memory-efficient attention kernels, and supports techniques like **RoPE scaling** for long contexts.
- It can leverage **DeepSpeed or FSDP for multi-GPU**, but even on a single GPU, its support for QLoRA means you can fine-tune, say, a 65B model on a 48 GB GPU (with quantization).
- One standout aspect is how quickly it incorporates new efficiencies: for example, when k-bit quantization algorithms or better optimizers appear in papers, LLaMA-Factory tends to include them.
- As a real-world reference, the fine-tuning *LLaMA-3 8B* on a single A100 40GB is completed in under 5 hours. On a smaller GPU like an RTX 3090 (24GB), one could still fine-tune 8B or 13B models with QLoRA, albeit slower (perhaps ~20 hours for the same job).

Example usage of LLaMA-Factory

For a code-based approach, LLaMA-Factory also exposes a Python API (in case you want to script things without YAML).

But typically, you'd edit a YAML, for instance, here's a snippet from a config for LoRA fine-tuning:

```yaml
# qwen3_lora_sft.yaml (excerpt)
model_name_or_path: Qwen/Qwen-3-4B
finetuning_type: lora
lora_rank: 8
dataset: path/to/dataset.json
template: llama2
per_device_train_batch_size: 2
gradient_accumulation_steps: 4
num_train_epochs: 3
output_dir: output/qwen3-4B-lora
```

Running `llamafactory-cli train` on this config would fine-tune the Qwen-3 4B model with LoRA rank 8 on your dataset.

After training, you could open the *Chat* tab in the WebUI, load the base model and the LoRA weights (it provides a UI element for selecting the adapter file), and have a conversation with the fine-tuned model to evaluate its responses.

DeepSpeed: Powering Large-Scale Fine-Tuning

DeepSpeed is a different beast in this lineup, it's not an all-in-one training app, but a **deep learning optimization library**.

Developed by Microsoft, DeepSpeed is renowned for enabling the training of **billion-parameter models** efficiently by overcoming hardware limitations.

It's the engine under many research projects and complements tools like Hugging Face's Trainer or PyTorch Lightning.

Usability

On its own, you typically use DeepSpeed by writing a Python training script and adding DeepSpeed initialization.

However for most users, the easiest way to leverage it is via **Hugging Face Accelerate/Trainer** integration.

For example, you can take a standard Transformers `Trainer` and simply supply a DeepSpeed config file (JSON), the Trainer will then use DeepSpeed under the hood for optimizations.

Alternatively, the **Accelerate** library can launch a distributed training with DeepSpeed by a simple config.

Here's an example accelerate config snippet for multi-node fine-tuning:

```
distributed_type: DEEPSPEED
num_machines: 2
mixed_precision: fp8
deepspeed_config:
  zero_stage: 3
```

In this configuration (used to fine-tune a LLaMA-70B in one case), `zero_stage: 3` turns on DeepSpeed's ZeRO-3 sharding, and `mixed_precision: fp8` uses **8-bit floating point** precision for even greater memory savings on supported GPUs.

With just this setup, you can call `accelerate launch` on your training script and it will automatically shard the model across 2 machines (each maybe with 4 GPUs), massively simplifying the orchestration.

For single-machine usage, you might use the `deepspeed` launcher script: e.g. `deepspeed --num_gpus=2 train.py --deepspeed ds_config.json`.

This would run your `train.py` with DeepSpeed wrapping your PyTorch model (assuming your script is DeepSpeed-compatible). In short, *DeepSpeed requires some coding or integration,* there's no GUI or simple

YAML for complete training (outside of the HF ecosystem).

But once set up, it's largely *"set and forget"* , you let DeepSpeed handle parallelism and memory.

DeepSpeed's flagship feature is the **ZeRO (Zero Redundancy Optimizer)**. ZeRO allows splitting the model's states (optimizer moments, gradients, and even parameters) across GPUs so that no single GPU needs a full copy.

This is how people fine-tune 30B, 65B, or larger models on a few GPUs. For example, with ZeRO-3, if you have 4 GPUs, each might hold only 1/4 of the model parameters in memory.

DeepSpeed also supports **ZeRO-Offload**, where some chunks of data (like optimizer states) can be offloaded to CPU RAM or even NVMe SSD, further extending the ability to train huge models on limited GPU memory (at the cost of speed).

Another core capability is **distributed training** across multiple nodes (not just multiple GPUs in one server, but across servers).

DeepSpeed integrates with communication backends to synchronize gradients efficiently. It's known to scale to thousands of GPUs for massive models.

DeepSpeed also introduced specialized optimizations:

- **Efficient Optimizers:** Like 1-bit Adam and 1-bit LAMB, which reduce communication overhead during multi-GPU training by quantizing gradient exchange (useful for distributed fine-tuning).
- **Mixed Precision Training:** It handles FP16 and BF16 seamlessly (automatically doing loss scaling, etc.). And now, it supports **FP8** (when using accelerate on hardware like NVIDIA H100, as shown above).
- **Memory-Efficient Attention and Long-Sequence Support:** DeepSpeed has a module called *Sparse Attention* and experimental features for very long sequence training (it introduced an "Arctic" optimization for long sequences), relevant if you fine-tune models on contexts beyond 2048 tokens.
- **MoE (Mixture of Experts):** If you fine-tune models with MoE layers (not common for most, but some research models use MoE), DeepSpeed has built-in support to efficiently train those by gating which expert is active per token.

However, note that DeepSpeed *itself* doesn't implement LoRA or QLoRA, but it works with them. For example, you can fine-tune a 70B with QLoRA by using Hugging Face PEFT to add LoRA layers to the model, and still use DeepSpeed ZeRO to handle the base model distribution and training.

This is a common setup: **DeepSpeed + QLoRA + HF Trainer**.

This combination can fine-tune LLaMA-70B on a **2x H100 (80GB) node** setup, using ZeRO-3, 4-bit quantization, and LoRA adapters.

The result is a production-ready pipeline where even such a large model can be fine-tuned in a reasonable time.

Without DeepSpeed, it would be near-impossible to handle the model purely in memory.

Performance

DeepSpeed has been shown to *increase throughput and reduce cost* for large model fine-tuning significantly.

DeepSpeed can enable 15x speedup in certain RLHF fine-tuning stages (compared to naive approaches). For most users fine-tuning moderate models (6B–13B), the benefit of DeepSpeed on a single GPU is mainly in memory saving (ZeRO-Offload), which might allow you to use a larger batch size or longer sequence without out-of-memory.

On multiple GPUs, it shines by achieving near-linear scaling as you add more GPUs.

That said, if you're just fine-tuning a 7B on one GPU, DeepSpeed might be overkill, something like Unsloth or Axolotl (with bitsandbytes) is simpler.

DeepSpeed comes into the picture when pushing limits: e.g. fine-tuning a **40B model on a 24 GB GPU** by offloading half to CPU, or training a **70B on 8x 16GB GPUs** by sharding.

It's also invaluable for multi-node clusters where writing custom distributed code would be complex, DeepSpeed handles it internally.

Example for Fine-Tuning with DeepSpeed (via HF Trainer):

To illustrate, here's how you might use DeepSpeed in a Transformers training script:

```
from transformers import Trainer, TrainingArguments, AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-70b-chat-hf")
# (wrap model with LoRA using PEFT if desired here)

training_args = TrainingArguments(
    output_dir="out-model",
    per_device_train_batch_size=1,
    gradient_accumulation_steps=8,
    fp16=True,
    deepspeed="ds_config_zero2.json"   # Path to DeepSpeed config
)
trainer = Trainer(model=model, args=training_args, train_dataset=my_data)
trainer.train()
```

With the `deepspeed` argument, when `trainer.train()` is called, it will initialize DeepSpeed. A simple DeepSpeed config (`ds_config_zero2.json`) might contain:

```
{
  "zero_optimization": {
    "stage": 2,
    "offload_optimizer": { "device": "cpu" }
  },
  "fp16": {
    "enabled": true
  }
}
```

This would use **ZeRO Stage 2** (shard optimizer states and gradients across GPUs) and offload the optimizer to CPU (saving GPU memory).

The result: your 70B model's gradients and optimizer live partly in CPU RAM, allowing the training to proceed on, say, 2 GPUs that each hold a shard of the weights. DeepSpeed handles all the syncing and swapping behind the scenes.

Axolotl: Configuration-Driven Fine-Tuning Made Fun

Axolotl is an open-source framework focused on simplicity and flexibility.

It's often described as *"fine-tuning with a single YAML config"*.

The philosophy is to let you specify *what* you want to fine-tune and *how* (which model, data, method, hyperparams) in a config file, and Axolotl's CLI will handle the rest, from preprocessing to training to saving the model.

Usability

Getting started with Axolotl typically involves installing it via pip or Docker, then using one of the example config files as a template for your task.

For instance, after installation you might run:

```
axolotl fetch examples
axolotl train examples/llama-3/lora-1b.yml
```

Copy

This will download a bunch of sample YAMLs (covering different models and methods). You can modify the YAML to point to your dataset or change hyperparameters.

The YAML configuration approach means you don't write code for each fine-tune, you just edit fields.

A config file typically includes sections like: model (pretrained model name/path), data (dataset paths or HuggingFace dataset names), training parameters (batch size, epochs, learning rate), LoRA parameters if applicable (rank, alpha), and any special settings (e.g. enabling FlashAttention, gradient checkpointing, etc.).

This design makes it **very reproducible,** you can share a config with someone else, and they can re-run the fine-tune exactly.

Axolotl lacks a GUI, but its console output is clear and it will periodically print training loss, etc.

It also supports logging to WandB if configured.

Because it's config-driven, Axolotl is great for running on cloud instances or headless servers (no need for notebooks or GUIs).

Features

Despite its light footprint, Axolotl is **feature-packed**:

- **Model Support:** It can fine-tune virtually any Hugging Face model. It mentions GPT-OSS, LLaMA, Mistral, Pythia, and many more. It also supports new models like **Kimi, Plano, InternVL 3.5, Olmo-3, Trinity**, etc.
- **Multi-modal and beyond-LLM:** Axolotl isn't limited to chat LLMs. It supports **vision-language models** (like LLaMA-Vision, Qwen-VL) and **audio models** (like speech recognition or TTS) with image/video/audio inputs. It even has an extension for **diffusion models fine-tuning** (text-to-image).
- **Training Methods:** You can do **full fine-tuning** or various parameter-efficient methods. It supports **LoRA and QLoRA**, with some unique optimizations.

- Axolotl supports **GPTQ** training (GPTQ is typically an inference quantization, but here it likely means you can load a GPTQ quantized model and maybe fine-tune or at least merge LoRA into it).
- Axolotl also supports **Quantization Aware Training (QAT)** to allow training models with quantization simulation for int4/int8 to improve final quantized accuracy.
- On the preference optimization side, it supports methods like **DPO, IPO, ORPO** (these are fine-tuning methods to align with preferences without RL). For **RL**, it has **GRPO** (Guidance Regularized Policy Optimization, a newer RLHF variant) support integrated. And for reward models, it can do reward model fine-tuning as well.

Optimizations

Under the hood, Axolotl bundles lots of tricks:

- **FlashAttention** and **xFormers** for faster attention
- **Liger** kernels (efficient fused kernels),
- sequence packing (they call it "Multipack") to utilize context space fully,
- **Sequence Parallelism**
- **Fully Sharded Data Parallel (FSDP)**
- DeepSpeed integration for multi-GPU.

Essentially, Axolotl tries to give you the benefits of DeepSpeed/Accelerate without you writing that code, you flip a switch in the config to turn on DeepSpeed ZeRO stage 2 or 3 for example.

It's like a meta-framework that orchestrates PyTorch, Transformers, PEFT, DeepSpeed, etc., based on the YAML.

Integrations

It can load datasets from local files, HF Hub, or even cloud buckets (S3, GCS). And when training is done, you can easily push the model to Hugging Face Hub (via the `huggingface_hub` library, by providing your token).

Axolotl also provides **Docker images** to encapsulate the environment, which is handy for ensuring consistency across setups.

Other features:

- Supports **FP8 fine-tuning** (via PyTorch's prototype FP8, enabled with `torch.ao` backend), making it one of the first open tools to allow FP8, which is great for new GPU owners.
- **N-D parallelism**, combining tensor, pipeline, and data parallel strategies in one (for extreme multi-GPU scaling).
- Support for a flurry of new research models like **Magistral** (Mistral variant), **Granite 4**, **HunYuan**, etc..
- It also implemented **sequence parallelism** for long context training (enabling context lengths beyond what fits on one GPU by splitting sequences across GPUs).
- You can also use Unsloth's packed data loader within Axolotl if desired.

Example for Axolotl YAML (LoRA fine-tune)

Suppose we want to fine-tune a Mistral 7B model on a custom instruction dataset using QLoRA.

We might create `mistral_qlora.yml`:

```yaml
base_model: lmsys/mistral-7b-v0.1
datasets:
  - path: data/my_instructions.json
    type: alpaca   # data format (Alpaca-style)
training:
  num_epochs: 2
  batch_size: 16
  lr: 2e-4
peft:
  method: qlora
  lora_r: 16
  lora_alpha: 32
  target_modules:
    - q_proj
    - v_proj
    # (which parts of the model to attach LoRA to)
hardware:
  mixed_precision: bf16
  deepspeed: "zero2"    # enable DeepSpeed ZeRO Stage 2
```

Then running `axolotl train mistral_qlora.yml` will commence the fine-tuning according to those settings.

Axolotl will handle converting the dataset, loading the model, applying LoRA, using DeepSpeed with BF16, etc.

The **target_modules** in LoRA config let us specify we only want LoRA on certain layers (common for Mistral and LLaMA to target Q and V linear layers).

This level of detail is configurable, whereas other tools might hardcode defaults.

Thoughts

Each framework and tool come with an authentic community and extensive documentation to help you achieve the best results in customizing LLMs for your product or research.

Happy fine-tuning!

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

GLM-4.7-Flash on 24GB GPU (llama.ccp, vLLM, SGLang, Transformers)

GLM-4.7-Flash is one of those rare open-weights releases that changes what "local-first" can realistically mean for...

medium.com

ClawRouter: Anthropic charged me $4,660 - How I cut it 70% with smart LLM routing

Anthropic charged me $4,660.87, just for Claude. Most requests were routed to the Claude Opus model. This is how I cut...

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic...

medium.com

Qwen3-Coder-Next Failed in the Scariest Way: 10 Practical Tests

Qwen3-Coder-Next Failed in the Scariest Way: 10 Practical Tests Benchmarks are the LinkedIn of LLMs. Every model looks...

medium.com

Why Codex Became My Default Over Claude Code (for Now)

If you haven't tried Codex yet, I've got a brief take that might save you a few hours.

medium.com

"This PDF hates OCR" - DeepSeek OCR 2 Reads Documents Like A Human

"This PDF hates OCR" - DeepSeek OCR 2 Reads Documents Like A Human Your customer forwards an invoice PDF with the...

medium.com

Vectorless RAG for Agents: PageIndex Is Why Their Demo Works and Yours Needs "Context"

When I'm building a RAG apps, the thing that quietly kills quality (and budgets) is turning a document into a pile of...

medium.com
#fine-tuning#fine-tuning-llm#llm-finetuning#model-finetuning#fine-tuning-transformer

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel